

Phase 1: System Architecture Design

CSE354 – Distributed Computing Project

1. Overview of the System

The system is a **distributed AI inference platform** designed to handle **1000+ concurrent LLM (Large Language Model) requests** efficiently using **GPU-based worker nodes**, a **load balancer**, a **master scheduler**, and a **RAG (Retrieval-Augmented Generation) module**.

The main objective is to ensure:

- Low latency response under heavy load
 - Efficient GPU utilization
 - Scalability to large number of users
 - Fault tolerance in case of node failure
-

2. High-Level Architecture

The system is composed of the following main components:

1. Client Layer (Load Generator)

- Simulates 100–1000+ concurrent users
- Sends AI requests (queries) to the system
- Measures response time and throughput

2. Load Balancer (LB)

- First entry point for all incoming requests
- Distributes requests to the Master Scheduler or directly to workers (depending on design)
- Implements scheduling policies:
 - Round Robin
 - Least Connections
 - Load-aware routing

3. Master Node (Scheduler / Controller)

- Central brain of the system
- Receives requests from Load Balancer
- Assigns tasks to GPU worker nodes
- Tracks system state (worker health, queue sizes)
- Handles task reassignment in case of failure

4. GPU Worker Nodes

- Execute LLM inference tasks
- Run parallel computations
- Return responses back to Master Node
- Report health status periodically

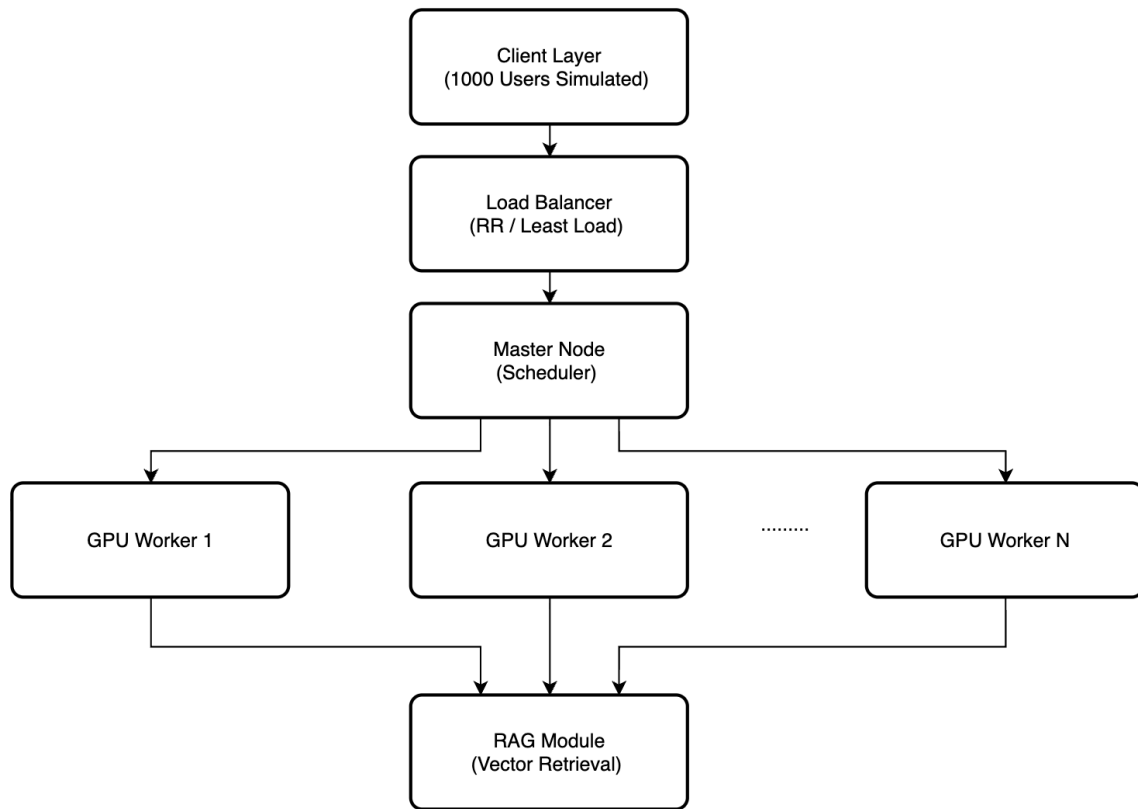
5. RAG Module (Retrieval-Augmented Generation)

- Retrieves relevant context from knowledge base (vector DB simulation)
- Enhances prompt before sending to LLM
- Improves response accuracy

6. LLM Inference Engine (Stub)

- Simulates heavy AI computation
 - Returns generated response based on input prompt + RAG context
-

3. System Architecture Diagram (Logical)



4. Data Flow Design

Step-by-step request flow:

1. **Client generates request**
 - Example: "Explain distributed systems"
2. **Request sent to Load Balancer**
 - LB selects a routing strategy
3. **Load Balancer forwards request to Master Node**
4. **Master Node processes request**
 - Checks worker availability
 - Selects best GPU node (based on load or RR)
 - Sends task to worker
5. **RAG module is triggered**
 - Retrieves context (documents / embeddings)
 - Enhances query
6. **GPU Worker executes LLM inference**
 - Processes enhanced prompt

- Generates response
 - 7. **Response returned**
 - Worker → Master → Client
-

5. Load Balancing Strategy Design

We implement and compare:

1. Round Robin

- Cycles through workers sequentially
- Simple and fair distribution
- Does NOT consider load differences

2. Least Connections

- Sends request to worker with fewest active tasks
- Better for uneven workloads

3. Load-Aware Routing (Advanced)

- Uses:
 - CPU/GPU utilization
 - Queue length
 - Chooses least loaded node dynamically
-

6. GPU Cluster Design

Each GPU worker node:

- Maintains a task queue
- Processes tasks asynchronously
- Supports parallel inference simulation
- Reports:
 - Load status
 - Active tasks
 - Failure state

Optimization Goals:

- Max GPU utilization
 - Min idle time
 - Balanced workload distribution
-

7. Fault Tolerance Design

7.1 Worker Node Failure Handling

- Master Node continuously monitors worker health
- If a worker fails:
 - Mark as inactive
 - Reassign pending tasks

7.2 Task Reassignment

- Each task has:
 - Task ID
 - Retry counter
- If failure occurs:
 - Task is re-queued in Master Node

7.3 Load Balancer Resilience

- Load Balancer maintains backup routing logic
 - If Master Node is slow/unavailable:
 - Temporary direct routing to healthy workers (fallback mode)
-

8. RAG Integration Design

Purpose:

Improve LLM response quality using external knowledge.

Process:

1. Receive user query
2. Convert query into embedding (simulated)

3. Search top-k relevant documents
4. Append context to prompt
5. Send enriched prompt to LLM

Example:

User Query:

"What is distributed computing?"

RAG Output:

Adds textbook definition + related notes before LLM processing

9. Technology Stack Selection

Component	Technology
Language	Python
Concurrency	threading / multiprocessing / asyncio
LLM Simulation	PyTorch / mock inference
Load Balancer	Custom Python + optional HAProxy/NGINX
GPU Simulation	CUDA-like stub or multiprocessing
RAG System	FAISS-like vector simulation
Communication	Queue / socket / RPC simulation

10. Scalability Design

System is designed to scale in:

Horizontal Scaling:

- Add more GPU worker nodes dynamically
- Load balancer auto-updates node pool

Vertical Scaling:

- Improve GPU power per node
- Increase batch size for inference

Request Scaling:

- Handle 100 → 1000+ concurrent users via:
 - Async processing
 - Task queues
 - Worker pooling
-

11. Assumptions

- GPU nodes are homogeneous (same capacity)
 - Network latency is negligible in simulation
 - LLM inference is mocked (not real model execution)
 - RAG uses simplified in-memory structure
-

12. Limitations (Phase 1 Design)

- No real GPU computation yet (simulation only)
 - No real distributed network protocol (e.g., gRPC)
 - No persistence layer for logs/results
 - RAG is simplified (not production vector DB)
-

13. Expected Outcomes

After implementation:

- System can simulate 1000+ concurrent requests
- Load is distributed across GPU workers
- Failures are detected and recovered automatically
- RAG improves response relevance
- Latency and throughput can be measured